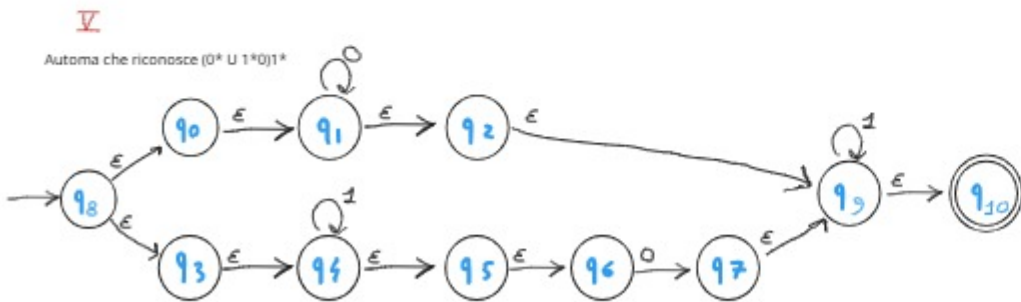
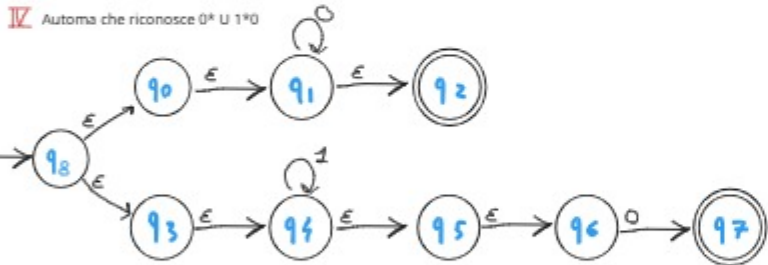
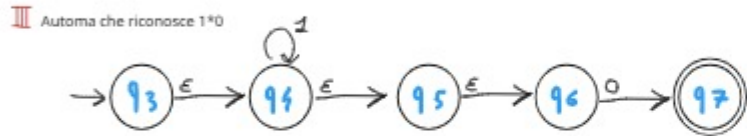


ESERCIZI PARTE 1

(16 punti) Data l'espressione regolare $E = (0^* \cup 1^*0)^*$, utilizzare il metodo studiato per costruire un automa finito $\mathcal{A}$ tale che $L(\mathcal{A}) = L(E)$. Illustrare e spiegare ogni passaggio fatto.

L'idea è quella di costruire l'automa partendo dalle sue componenti più piccole e dalla parentesi più interne



16 Luglio 2025

8

7. Sia $L = \{xycy \mid x, y \in \{a, b\}^*, x \text{ e } y \text{ hanno lo stesso numero di } a \text{ ma non lo stesso numero di } b\}$. Ad esempio, $abacba, abacab \in L$ ma $acba, abacab \notin L$. Provare o confutare che L è regolare.

Supponendo per assurdo che L sia regolare, abbiamo p come costante del pumping lemma. Sappiamo quindi che per ogni w in L con $|w| \geq p$, possiamo dividere w in 3 sottostringhe x, y e z tali che $w = xyz$. Sappiamo che valgono le seguenti proprietà:

$$|xy| \leq p; |y| > 0; xy^iz \in L \quad \forall i \geq 0$$

Prendiamo una stringa appartenente ad L , cioè $w = (a^k c a^j b)$. La stringa vi appartiene poiché presenta un numero p di a da ambo i lati ed un numero diverso di b , 0 a s_x e 1 a s_y . Verifichiamo ora che le proprietà siano rispettate:

$$x = a^k, k \geq 0 \quad k + j \leq p$$

$$y = a^j, j > 0 \quad z = c a^p b$$

$$xy^iz \in L$$

(Strettamente, poiché j era almeno 1)

$$i=0 \Rightarrow w^1 = xz = a^k c a^p b \notin L, k < p$$

Un'espressione regolare (regex) è un modo dichiarativo per descrivere un linguaggio regolare.

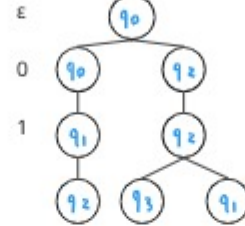
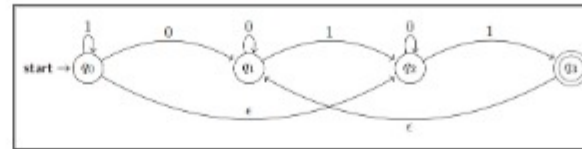
Dati R_1 ed R_2 regex, rispettivamente, dei linguaggi L_1 ed L_2 , definiamo le operazioni:

- Unione $R_1 \cup R_2$ rappresenta $L_1 \cup L_2$
- Concatenazione $R_1 R_2$ rappresenta $L_1 L_2$
- Chiusura di Kleene o Kleene Star R_1^* rappresenta L_1^*

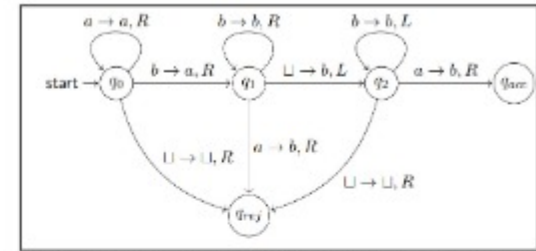
Il Teorema di Kleene afferma che un linguaggio L è regolare se e solo se ammette una regex.

L'idea della dimostrazione è la seguente:

- Sappiamo che L è regolare $\Leftrightarrow L$ è riconosciuto da un NFA $\Leftrightarrow L$ è riconosciuto da un DFA.
- Vogliamo mostrare che per ogni DFA A possiamo costruire una regex R tale che $L(A) = L(R)$.
- Ciò implica che per ogni regex R esisterà un NFA N tale che $L(N) = L(R)$.
- Possiamo quindi concludere dicendo che L è riconosciuto da un DFA $\Leftrightarrow L$ ammette una regex, da cui L è regolare $\Leftrightarrow L$ ammette una regex.

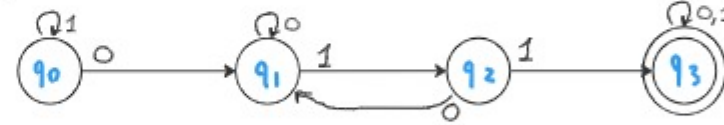


La computazione della stringa '01' termina ed uno dei rami giunge in uno stato accettante, dunque la stringa è accettata. Se non fossimo arrivati in q_3 o non avessimo potuto muoverci perché l'input non portava in nessuno stato per i di un certo livello, allora avremmo rifiutato.



Su input bbb si ha la seguente computazione (uso Q per leggibilità, ma è minuscola, ed U come blank): $Q0bbb \rightarrow aQ1bb \rightarrow abQ1b \rightarrow abbQ1U \rightarrow abQ2bb \rightarrow aQ2bbb \rightarrow Q2abbb \rightarrow bQACbb$

Dare un DFA che riconosca il linguaggio su $\Sigma = \{0, 1\}$ formato da tutte le stringhe che hanno 011 come sottostringa.



È possibile che venga chiesto di costruire la regex dato il linguaggio, ma in 3 casi su 4 si tratta della seconda richiesta dell'esercizio; la prima vuole che venga costruito il DFA.

È fortemente consigliato passare comunque per l'automa in quell'altro caso, poiché si rischia di dimenticare dei pezzi.

L'idea, in ogni caso, consiste nell'individuare i casi base del linguaggio, diciamo opzioni A, B e C, e fare questo: $((A)^* \cup (B)^* \cup (C)^*)^*$

1. Sia A un automa finito deterministico la cui funzione di transizione è riportata di seguito (la freccia indica lo stato iniziale e l'asterisco lo stato finale). Determinare (applicando la metodologia studiata) l'espressione regolare R tale che $L(A) = L(R)$.

	0	1
$\rightarrow q_0$	q_1	q_1
q_1	q_1	q_2
$* q_2$	q_1	q_2
q_3	q_1	q_3

Nel caso qualche sotto-espressione diventi troppo lunga, è sempre possibile sostituirla con un generico carattere x . La conversione da NFA a regex è sostanzialmente identica



Prima Iterazione

Aggiungiamo S prima dello start con una epsilon transition ed F dopo ogni stato finale, in modo che sia l'unico nuovo accettante. Aggiungiamo $*$ alle transizioni che portano in se stessi ed una unione nel caso vi siano più simboli.

Terza Iterazione

Rimuoviamo q_1 , notando che l'unico percorso da considerare è una strada alternativa per andare da q_0 a q_2 .

Seconda Iterazione

Rimuoviamo q_3 senza impattare nulla, poiché non si va da nessuna parte passando per lui. Fungeva soltanto da pozzo del DFA per rifiutare le stringhe.



Quarta Iterazione

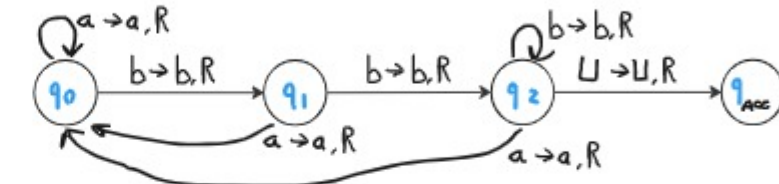
Rimuoviamo q_2 , notando che c'è da considerare soltanto il loop che presentava.

Quinta Iterazione

Rimuoviamo q_0 , senza modificare ulteriormente l'espressione trovata, che è la regex richiesta.

3. Fornire il diagramma di stato di una macchina di Turing deterministica che decide il linguaggio

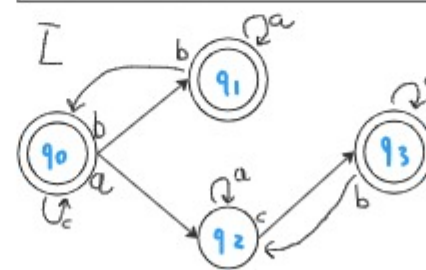
$$L = \{wbb \mid w \in \{a, b\}^*\}$$



Le transizioni non indicate portano ad uno stato di rifiuto

(16 punti) Sia $L = L(\mathcal{A})$ il linguaggio riconosciuto dall'automa finito $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$, dove $Q = \{q_0, q_1, q_2, q_3\}$, $\Sigma = \{a, b, c\}$, $F = \{q_2\}$ e δ è descritta dalla tabella seguente. Determinare il diagramma di una macchina di Turing M tale che $L(M) = \bar{L}$. È necessario spiegare il ragionamento fatto.

	a	b	c
q_0	$\{q_2\}$	$\{q_1\}$	q_0
q_1	$\{q_1\}$	$\{q_0\}$	$\emptyset$
q_2	$\{q_2\}$	$\emptyset$	$\{q_3\}$
q_3	$\emptyset$	$\{q_2\}$	$\{q_3\}$



Il diagramma della TM si costruisce in così:

- Le transizioni sono scritte così per ogni x appartenente a Σ : $x \rightarrow x, R$
- Gli stati accettanti, cioè q_0, q_1 e q_3 portano a q_{ACC} se viene letto un blank, con una freccia che segue sempre lo schema scritto sopra.
- Ogni altra transizione non esplicitata porta ad un rifiuto.